**UNIVERSIDAD SERGIO ARBOLEDA**

Escuela de Ciencias Exactas e Ingeniería

Ingeniería en Ciencias de la Computación e Inteligencia Artificial (ICCIA)

# Documento de Arquitectura y Especificación UML

Entrega Parcial (Sesión 3) - Entregable 1

Paradigmas de Programación (SIST0176) - Semestre III

**Grupo 4:** Segmentación y Procesamiento de Imágenes Médicas por Visión Artificial

Integrantes: [Nombre 1], [Nombre 2], [Nombre 3], [Nombre 4] (completar)

Docente: [completar] | Fecha: noviembre de 2026

## 1. Alcance y caso de estudio

**Problema (Grupo 4).** Un centro de diagnóstico necesita procesar tomografías (TAC) de alta resolución, aplicar filtros de convolución bidimensional para detectar bordes de tejidos y segmentar automáticamente regiones tumorales.

**Solución multiparadigma.** Tres módulos conectados, cada uno en el paradigma y lenguaje exigidos por la especificación: Módulo 1 imperativo en C++17 con OpenMP (convolución sobre memoria contigua), Módulo 2 orientado a objetos en Java 17 (modelo de dominio, Factory Method, DIP, copias defensivas) y Módulo 3 funcional en Python 3.10+ (mapeo puro de intensidades, recursión de cola y contenedor Option).

## 2. Notación usada

| Símbolo | Significado |
|---|---|
| + / - / # / ~ | Visibilidad: público / privado / protegido / paquete |
| Línea continua con triángulo hueco | Generalización (herencia, *extends*) |
| Línea discontinua con triángulo hueco | Realización (*implements* de una interfaz) |
| Línea con rombo hueco | Agregación: el todo referencia a las partes, que viven por separado (inyección de dependencias) |
| Línea con rombo relleno | Composición: el todo es dueño de las partes y las destruye con él |
| Flecha discontinua abierta | Dependencia (usa, crea, lanza). Estereotipos <<use>>, <<create>>, <<throws>> |

# 3. Diagrama de clases UML - Módulo POO (Java 17)

## 3.1 Dominio, repositorio y servicio

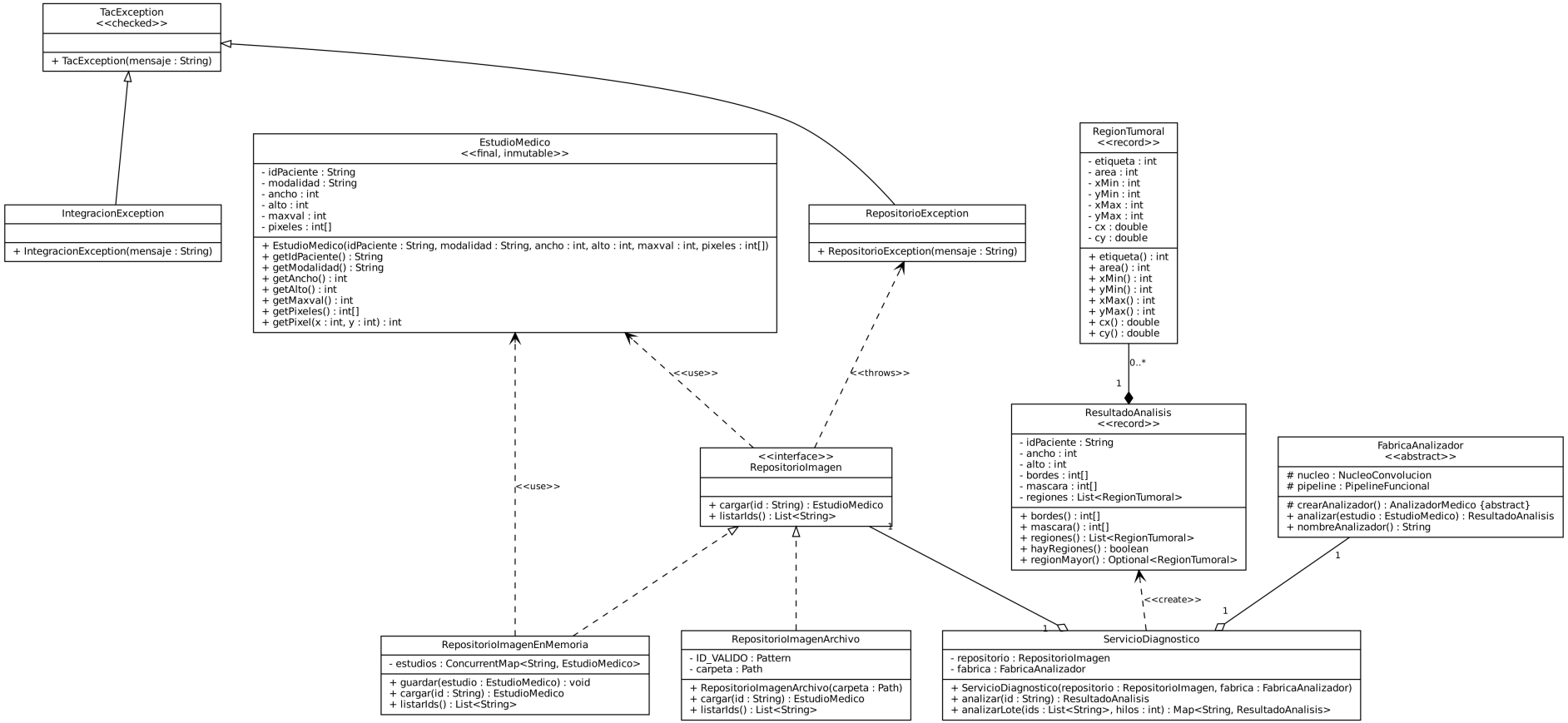



*Figura 1. Dominio, repositorio, servicio y excepciones (Generalización, Realización, Agregación, Composición, Dependencia).*

## 3.2 Factory Method y Template Method (análisis)

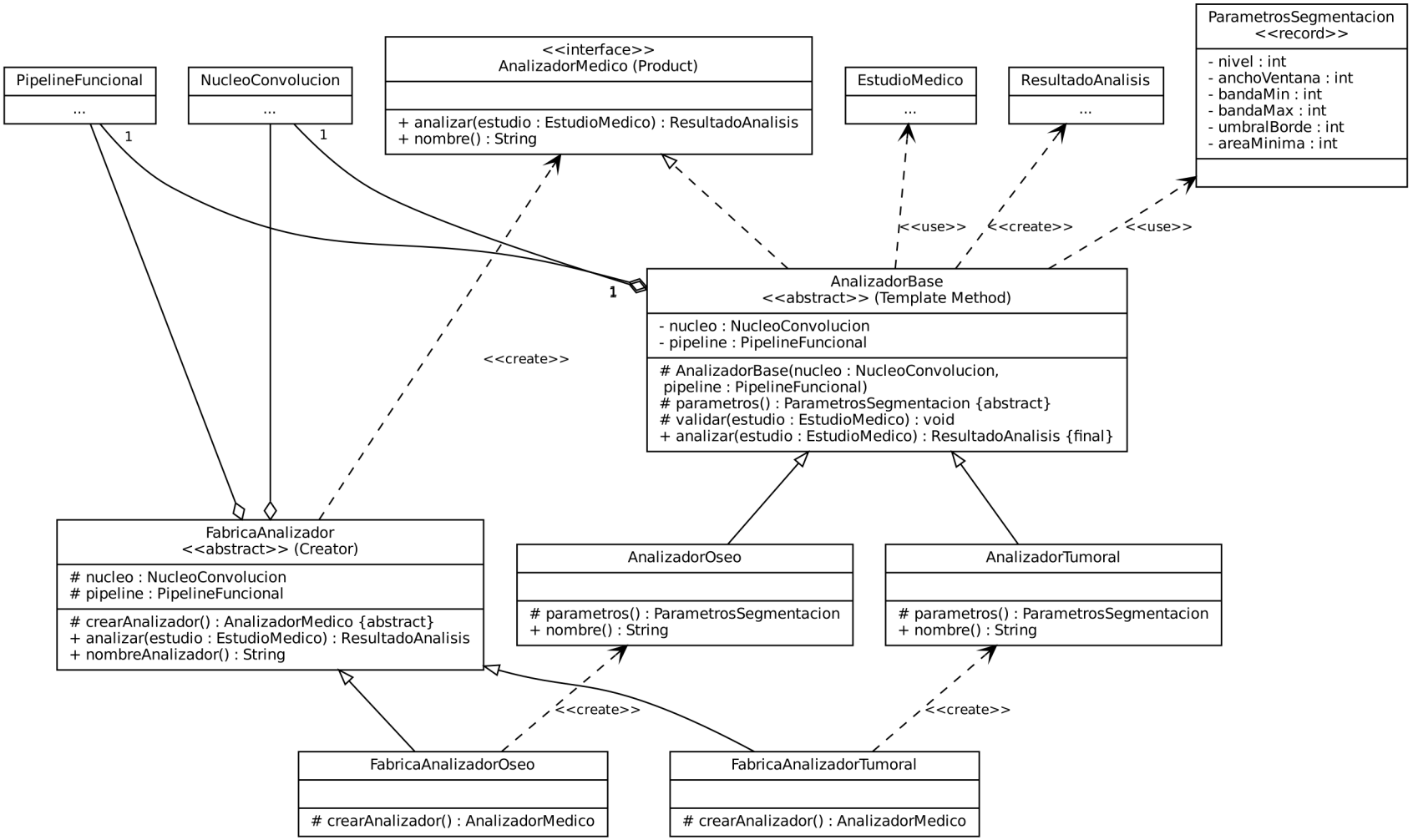



*Figura 2. Fábricas (Creator), analizadores (Product) y clase abstracta con método plantilla.*

## 3.3 Integración con los módulos C++ y Python

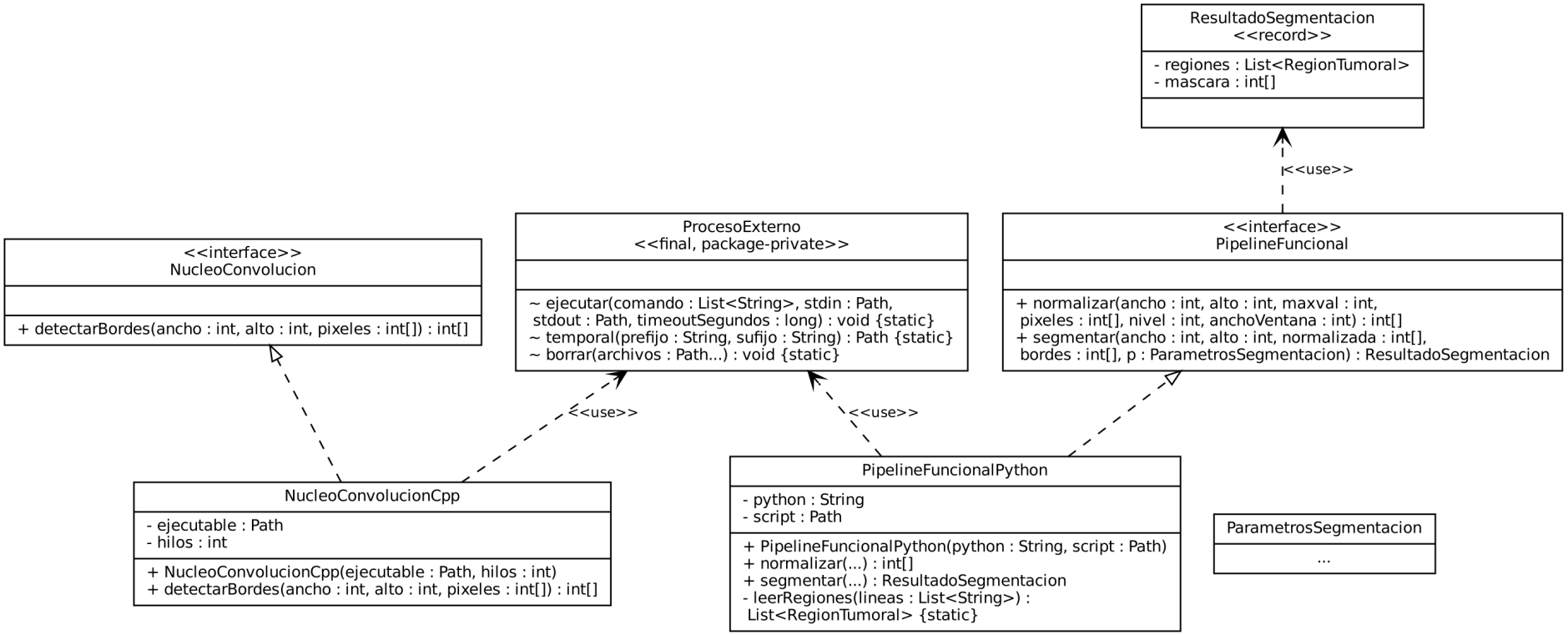



*Figura 3. Contratos (interfaces) y adaptadores hacia el kernel C++ y el pipeline funcional.*

# 4. Diagrama de flujo de datos e integración entre módulos

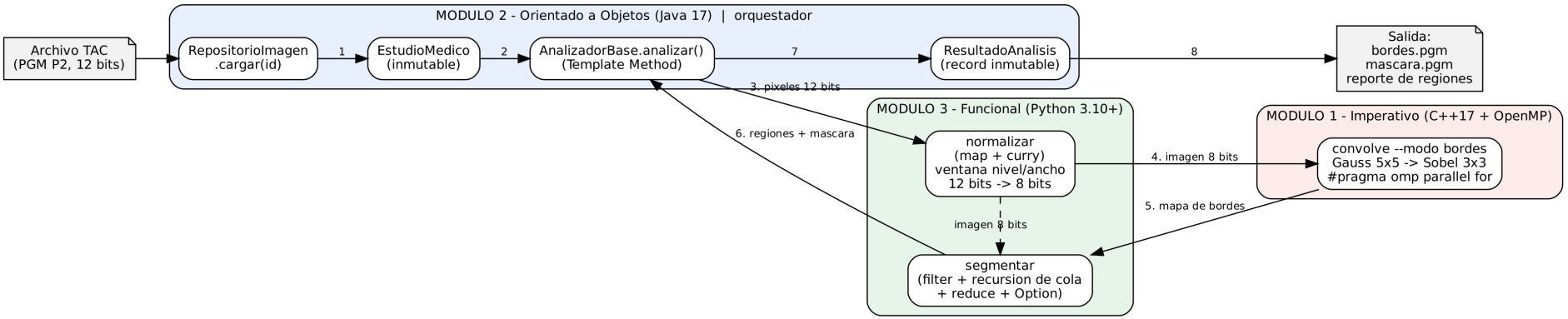



*Figura 4. Flujo de datos. Java orquesta; Python normaliza y segmenta; C++ calcula bordes.*

**Formato de intercambio.** Las imágenes viajan entre módulos como archivos PGM P2 (texto), que los tres lenguajes leen sin librerías externas. Java ejecuta los otros módulos como procesos y redirige su entrada/salida a archivos temporales únicos, lo que evita bloqueos por buffers y permite llamadas concurrentes seguras.

## 5. Contratos de interfaz entre módulos

| Contrato | Entrada | Salida | Quién lo implementa |
|---|---|---|---|
| NucleoConvolucion.detectarBordes | ancho, alto, píxeles 8 bits | Mapa de bordes 0..255 | NucleoConvolucionCpp (llama a build/convolve) |
| PipelineFuncional.normalizar | píxeles 12 bits, nivel, ancho de ventana | Píxeles 8 bits | PipelineFuncionalPython (cli.py normalizar) |
| PipelineFuncional.segmentar | imagen 8 bits, bordes, ParametrosSegmentacion | ResultadoSegmentacion (regiones + máscara) | PipelineFuncionalPython (cli.py segmentar) |
| RepositorioImagen.cargar | id de estudio | EstudioMedico inmutable | RepositorioImagenArchivo / RepositorioImagenEnMemoria |

## 6. Justificación de diseño: SOLID y patrones GoF

| Principio / patrón | Dónde se aplica | Justificación |
|---|---|---|
| S - Responsabilidad única | EstudioMedico, FormatoPGM, ProcesoExterno, ServicioDiagnostico | Cada clase tiene un solo motivo de cambio: modelar un estudio, leer PGM, ejecutar procesos, orquestar el caso de uso. |
| O - Abierto/Cerrado | FabricaAnalizador y AnalizadorBase | Para un nuevo tipo de análisis (p. ej. vascular) se agrega una subclase de analizador y una de fábrica; no se modifica código existente. |
| L - Sustitución de Liskov | AnalizadorTumoral / AnalizadorOseo | Ambos pueden usarse donde se espera AnalizadorMedico: respetan el contrato y no endurecen precondiciones. |
| I - Segregación de interfaces | NucleoConvolucion y PipelineFuncional | Interfaces pequeñas y separadas, una por módulo externo. |
| D - Inversión de dependencias | ServicioDiagnostico, AnalizadorBase | Dependen de interfaces (RepositorioImagen, NucleoConvolucion, PipelineFuncional), no de C++/Python/archivos. Las pruebas usan falsos sin esos módulos. |
| GoF: Factory Method | FabricaAnalizador.crearAnalizador() | Las subclases deciden qué analizador instanciar; el servicio no conoce clases concretas. |
| GoF: Template Method | AnalizadorBase.analizar() (final) | Fija el flujo normalizar -> bordes -> segmentar; las subclases solo definen parametros(). |
| Inmutabilidad / copias defensivas | EstudioMedico, records de resultados | Se copia el arreglo al entrar y al salir; List.copyOf produce listas inmodificables: no hay aliasing. |